

```
# Run the app when container starts
```

```
CMD ["python", "app.py"]
```

```
Using Docker CLI
```

```
# List running containers
```

```
docker ps
```

```
# List all containers (running and stopped)
```

```
docker ps -a
```

```
# Stop a container (replace CONTAINER_ID with actual ID)
```

```
docker stop CONTAINER_ID
```

```
# Remove a container
```

```
docker rm CONTAINER_ID
```

```
# Remove an image
```

```
docker rmi IMAGE_NAME
```

```
# View logs of a container
```

```
docker logs CONTAINER_ID
```

```
# Enter a running container interactively
```

```
docker exec -it CONTAINER_ID /bin/bash
```

Limitations and considerations

Although Docker has altered the ways in which software is developed and deployed, there are some limitations and considerations to keep in mind while adopting it.

For one, security can be an issue since the containers share the host operating system's kernel. This matrix sharing of resources can increase the overall attack surface if containers are poorly isolated or configured. Organisations should follow best security practices, such as using minimal base images, running containers with least privilege, and regular vulnerability scans.

Performance-wise, the overhead is generally low for containers compared to bare metal applications, though it does exist. If an application has extremely high I/O or specialised hardware requirements, this overhead can

become noticeable.

Another major consideration is data persistence. Since containers are ephemeral in nature and can be spun up or destroyed at a moment's notice, there is one more variable to consider when persisting data -- the application side of Docker volumes and bind mounts -- leading to complexity in storage management.

Docker networking has its complexities when containers must cross multiple hosts or large multi-container applications are to be managed. Setting up container networks and assuring communication are complex endeavours, involving quite a steep learning curve.

When not to use Docker

Despite its advantages, Docker may not be suitable in some situations. GUI-intensive or desktop applications are one example. Containers truly run best when it comes to server-side and CLI applications, while graphical apps inside containers are rather poor performers and suffer from integration issues.

Applications requiring full OS feature sets or custom kernels are another case where Docker falls short. Since containers share the OS kernel of the host, apps that require custom kernel modules or deeper integration with the OS cannot really run inside containers.

Docker can seriously add unnecessary weight and complexity to very simple or small projects, especially those that don't have any complicated deployment needs. In these special scenarios, more traditional ways of deployment may be simpler and more effective.

Legacy applications with complicated state management or licensing constraints may run into difficulties with containers, either in terms of technical requirements or the legal constraints. **END** 🐧

 By: Sourav Kairi

The author is actively contributing to various AI and ML projects at a Tier 1 IT company. He is also a member of the Board of Studies (BOS) at a well-known engineering college. This reflects his dedication to both tech innovation and academic excellence.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU